

Project Executable Files

Date	30 June 2026
Team ID	
Project Name	Rising water
Maximum Marks	3 Marks

Project Executable Files

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	Yes
5	Environment configuration file (.env.example or similar)	Yes
6	Deployed application URL (if hosted)	https://rising-water-kj4c.onrender.com/
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA
8	Dockerfile / Containerization config (if applicable)	NA
9	Test files and test results	Yes
10	Demo video or walkthrough (if required)	Yes

/project-root

|

— app.py	- Main Flask application and API routing
— train.py	- ML pipeline: preprocessing, SMOTE, model training
— main.py	- Standalone model testing script
— locustfile.py	- Locust load testing configuration
— requirements.txt	- Python dependencies list
— .env	- Environment variables (Database URL, Secret Key)
— .env.example	- Template for environment variable setup
— /data	- Raw and processed datasets
— flood_dataset.xlsx	- Original flood dataset
— flood_dataset_expanded.csv	- Expanded training dataset
— /models	- Serialized ML artifacts
— flood_model.pkl	- Trained XGBoost inference model
— preprocessor.pkl	- StandardScaler for data transformation
— feature_names.pkl	- Saved feature column names
— /reports	- Model evaluation outputs
— classification_report.txt	- Precision, Recall, F1-Score
— confusion_matrix.png	- Confusion matrix visualization
— feature_importance.png	- Feature importance bar chart
— roc_curve.png	- ROC-AUC curve plot
— /templates	- Jinja2 HTML view templates
— home.html	- Landing page
— index.html	- Prediction input form
— chance.html	- HIGH risk result page
— no_chance.html	- LOW risk result page
— history.html	- Prediction history dashboard
— reports.html	- ML model evaluation metrics page
— login.html	- User login form

└─	register.html	- User registration form
└─	/static	- Frontend assets
└─	home.css	- Primary glassmorphism stylesheet
└─	main.css	- Secondary component styles
└─	main.js	- Client-side validation and interactions
└─	background.png	- UI background image

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	https://rising-water-kj4c.onrender.com
Login Credentials (Demo)	review@gmail.com password:12345
Platform / Hosting Provider	Render
Repository Link	https://github.com/gireesh2658/Rising_water
Demo Video Link	https://drive.google.com/file/d/1dsGe4Yv0zvR9iQF3FjtyAxspFWRhO5JT/view?usp=sharing

Step 4: Run Instructions **Pre-**

requisites:

- Python 3.12 or higher installed.
- PostgreSQL installed and running locally (or a remote Supabase/Render database URL).

Step 1: Clone the repository and navigate to the project directory: `cd "5. Project Development Phase"`

Step 2: Install all required backend dependencies: `pip install -r requirements.txt`

Step 3: Configure environment variables: Create a `.env` file in the root directory and add your database URL and secret

key: SQLALCHEMY_DATABASE_URI=postgresql://user:password@localhost:5432/db_name FLA
 SK_SECRET_KEY=your_secure_key

Step 4: Start the backend application (database tables will auto-generate on startup): python app.py

Step 5: Open your web browser and navigate to: http://localhost:5000

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Remote Database Connection Timeouts	Resolved. SQLAlchemy pool_pre_ping=True and pool_recycle implemented in app.py to seamlessly handle idle connection drops.
2	"Cold Start" on First Prediction	Expected Behavior. The very first API call takes ~0.5s longer as flood_model.pkl is loaded from disk into RAM. Subsequent predictions are instantaneous.
3	Browser Caching Stale CSS	Workaround: Evaluators should use Ctrl + F5 (Hard Refresh) if UI styling appears broken during initial setup.